

MongoDB : La base de données NoSQL

Flexibilité, Scalabilité et Performance pour vos applications.

Introduction aux concepts et méthodes CRUD

Structure des données : SQL vs MongoDB

Concept	Monde Relationnel (SQL)	Monde NoSQL (MongoDB)
Conteneur	Base de données	Base de données
Structure	Table	Collection
Enregistrement	Ligne (Row)	Document
Donnée	Colonne (Column)	Champ (Field)
Relation	Jointure (Join)	Document imbriqué

Le Document MongoDB (BSON)

Format BSON

Extension binaire de JSON pour stocker plus de types (dates, binaires).

Schéma Dynamique

Chaque document peut avoir sa propre structure.

Clé Primaire `_id`

Champ unique `_id` généré automatiquement.

```
{
  "_id": ObjectId("60d5f..."),
  "nom": "Jean Dupont",
  "age": 28,
  "adresse": {
    "ville": "Paris",
    "cp": "75001"
  },
  "tags": ["développeur", "expert"]
}
```

Opérations CRUD : Create (Créer)

insertOne()

Insère un **seul document** dans une collection. Si la collection n'existe pas, elle est créée automatiquement.

```
// Insertion d'un utilisateur  
db.users.insertOne({  
  nom: "Alice",  
  age: 25,  
  ville: "Lyon"  
});
```

insertMany()

Insère **plusieurs documents** en une seule requête. C'est plus performant pour les insertions en masse.

```
// Insertion groupée  
db.users.insertMany([  
  { nom: "Bob", age: 30 },  
  { nom: "Charlie", age: 22 }  
]);
```

Opérations CRUD : Read (Lire)

find(query)

Retourne un curseur sur les documents correspondant.

findOne(query)

Retourne le premier document correspondant.

Opérateurs de comparaison

\$gt : >	\$gte : >=
\$lt : <	\$lte : <=
\$ne : !=	\$in : liste

```
// Utilisateurs de 18 ans et plus  
db.users.find({ age: { $gte: 18 } });
```

```
// Trouver par nom  
db.users.findOne({ nom: "Alice" });
```

```
// Utilisation de $in  
db.users.find({ ville: { $in: ["Paris", "Lyon"] } });
```

Opérations CRUD : Update (Mettre à jour)

`updateOne(query, update)`

Modifie le premier document qui correspond aux critères de recherche. Idéal pour les mises à jour ciblées par identifiant unique.

`updateMany(query, update)`

Applique les modifications à tous les documents correspondant aux critères. Utile pour les mises à jour de masse.

L'opérateur `$set`

Indispensable pour modifier uniquement les champs spécifiés sans remplacer l'intégralité du document.

```
// Mise à jour d'un seul utilisateur  
db.users.updateOne(  
  { nom: "Alice" },  
  { $set: { age: 26 } }  
);
```

```
// Mise à jour de plusieurs documents  
db.users.updateMany(  
  { ville: "Paris" },  
  { $set: { statut: "actif" } }  
);
```

Opérations CRUD : Delete (Supprimer)

deleteOne(query)

Supprime le **premier document** qui correspond aux critères de recherche. Recommandé pour les suppressions précises par identifiant unique.

```
// Supprimer un utilisateur
db.users.deleteOne({
  _id: ObjectId("60d5f...")
});
```

deleteMany(query)

Supprime **tous les documents** correspondant aux critères. Utile pour nettoyer des données obsolètes ou groupées.

```
// Supprimer les inactifs
db.users.deleteMany({
  statut: "inactif"
});
```

ATTENTION : Une requête vide {} dans deleteMany() videra l'intégralité de la collection.

Indexation et Performance

Importance des index

Accélèrent fortement les lectures en évitant le parcours complet de la collection.

`createIndex(fields)`

Crée un index sur un ou plusieurs champs; peut garantir l'unicité.

Analyse avec `.explain()`

Permet de vérifier le plan d'exécution et l'utilisation des index.

```
// Créer un index sur l'email
db.users.createIndex( { email: 1 }, { unique: true } );

// Analyser une requête
db.users.find({ email: "alice@example.com"
}).explain("executionStats");
```


Framework d'Agrégation

\$match

Filtre les documents (similaire à WHERE en SQL).

\$group

Regroupe les documents par une clé spécifiée.

\$sort

Trie les documents résultants.

\$project

Sélectionne ou transforme les champs à afficher.

Le pipeline traite les données par étapes successives.

```
// Calculer la moyenne d'âge par ville
db.users.aggregate([
  { $match: { statut: "actif" } },
  { $group: {
    _id: "$ville",
    moyenneAge: { $avg: "$age" }
  } },
  { $sort: { moyenneAge: -1 } }
]);
```

Conclusion et Avantages de MongoDB

Développement Agile

Le schéma flexible s'adapte aux changements de besoins sans migrations complexes.

Performance Élevée

Le format BSON et l'indexation permettent des lectures et écritures massives à faible latence.

Scalabilité Native

Conçu pour l'évolutivité horizontale (sharding) et la haute disponibilité (replica sets).

Écosystème Complet

Outils comme MongoDB Atlas, Compass et drivers pour la plupart des langages modernes.